Ansible for beginners: Get started with IT automation

Apr 16, 2021 - 7 min read

Erin Schaffer

Ansible is a simple, powerful, and agentless tool that simplifies the process of IT automation and expedites DevOps efforts. Ansible works to help you automate and configure your infrastructure to save time and increase productivity. It's straightforward, secure, and powerful, making it an easy tool to learn and implement within your organization. Today, we'll discuss what Ansible is, what it does, key terms, and how to get started.

We'll cover:

Get hands-on with Ansible

In this course, you'll learn the ins and outs of Ansible to help you manage and automate your infrastructure and code deployment.

Ansible: Zero to Production Ready

What is Ansible?

Ansible is an **open-source orchestration and automation tool** used for **software provisioning, configuration management, and application deployment**. Ansible was first developed in 2012 by Michael DeHaan, the creator of Cobbler and Func. The company funding Ansible was acquired by RedHat, Inc. in 2015. RedHat was acquired by IBM in 2019, so now, Ansible lives under the IBM umbrella.

Ansible runs on Windows and Unix-like operating systems, providing infrastructure as code. It has its own declarative programming language for management and system configuration. It can **connect with cloud environments** like Amazon AWS and Microsoft Azure to help you manage and automate your infrastructure and code deployment.

Ansible is simple to install, connects easily to clients, and contains many features. It's **push-based and connects to clients via SSH**, so it doesn't require an agent on the client. By pushing modules to clients, the modules on the client execute locally and the outputs are pushed back to the Ansible server. It uses SSH-Keys to simplify the process of connecting the clients. Hostnames, IP addresses, and SSH ports are stored in inventory files. Once an inventory file is created and populated, Ansible can use it.

Benefits of Ansible for DevOps

Ansible is a preferred tool in DevOps organizations because it **streamlines automation and flexibility**. Some of the key benefits include:

- Open-source, free to use
- Doesn't require special system administration skills for installation and use
- Highly customizable
- Consistent and lightweight
- Very secure due to agentless abilities and use of OpenSSH security
- Comprehensive documentation
- Smooth learning curve
- Built with Python, one of the fastest and most robust programming languages
- Version control and configuration management
- Reliable deployments

Common use cases of Ansible

Ansible is a powerful tool that has a **wide variety of applications** and uses, particularly for connecting it to Docker environments or cloud services like AWS. Some of the main use cases of Ansible include:

- Configuration management
- Application development and deployment
- Software and infrastructure
- IT, security, and network automation
- Manage virtual machines in bulk to ensure each VM has the same configuration
- Define the configuration of your server running in the cloud so others can easily read it and use it
- Use Ansible Tower or AWX to create a graphical user interface

Key Ansible terms

Now that we've discussed the benefits and common use cases of Ansible, let's ge familiarized with some of Ansible's key terms.

- **Ansible server:** The machine with Ansible installed, which runs all tasks and playbooks
- **Hosts:** The devices you manage with Ansible
- **Playbook:** A framework where Ansible automation tasks are defined (written in YAML)
- **Play:** The execution of a playbook
- **Modules:** A command or set of commands made for execution on the client-side
- **Task:** A section that contains a single procedure you want to be executed
- **Tag:** A name you can assign to a task
- **Handler:** A task that is only called if a notifier is present
- **Notifier:** A section assigned to a task that calls a handler if the output is changed
- **Inventory:** A file containing Ansible client-server data
- **Fact:** Information retrieved from the client from global variables with the gather-facts operation

Getting started with Ansible

Now it's time to learn about some fundamentals of the tool. We'll walk you through getting started with Ansible, Ansible playbooks, and Ansible ad-hoc commands.

Remember that with Ansible, you have an Ansible server and hosts. The Ansible server is the machine where Ansible is installed, and the hosts are the machines handled by Ansible via the Ansible server. The Ansible server **can handle multiple hosts**.

Ansible server requirements:

- Python 2 (2.7) or 3 (3.5 or higher)
- Ansible doesn't support Windows as a control node, but you can use WSL to set it up on Windows 10

Host requirements:

- Python 2 (2.7) or 3 (3.5 or higher)

> **Note:** The way you set up your Ansible environment is dependent upon your device's requirements. Your next steps would be to download Ansible, then configure and automate your experience.

Ansible playbooks

Playbooks are the **foundation of configuration management and orchestrated automation** with Ansible. They are where you create instructions to determine the tasks you want to execute within your different hosts.

Playbooks are **reusable and repeatable**, making it easy to perform a task more than once. Playbooks are written in YAML and have a small amount of syntax. Each playbook is composed of one or more plays.

Each play defines two things at a minimum:

- One or more tasks to execute
- The hosts you want to target

A play begins by defining the `hosts` line, which is a list of one or more host patterns or groups divided by colons. The `hosts` line is followed by a `tasks` list. Ansible will execute the tasks on the designated hosts in order, one at a time.

For example, take a look at the playbook below:

```
1  ---
2  - hosts: webservers
3          tasks:
4          - name: deploy code to webservers
5                  deployment:
```

```
5                 deployment:
6                         path: {{ filepath }}
7                         state: present
8  - hosts: dbserver
9          tasks:
10         - name: update database schema
11           updatedbschema:
12                 host: {{ dbhost }}
13                 state: present
14 - hosts: webservers
15         tasks:
16         - name: check app status page
17                 deployment:
18                         statuspathurl: {{ url }}
19                         state: present
```

In this playbook example, we executed three plays: we deployed code to the web servers, updated the database schema, and checked the app status page.

Ad-hoc commands

Ad-hoc commands are another important part of the Ansible environment. They are used to **automate a single task** on one or more hosts.

They offer a fast and simple way to automate, but they're not meant for reusability. Ad-hoc commands are useful for occasional tasks. For example, if you wanted to restart a service on multiple hosts, you can easily achieve this with a single ad-hoc command.

Use cases for ad-hoc commands:

- Connectivity testing
- Fact gathering
- Managing files and/or services

Different ad-hoc command types and what they do:

- `file`: Add and/or remove directories
- `ping`: Test connectivity
- `stat`: Retrieve facts about directories

- `copy`: Copy files
- `replace`: Update files
- `debug`: Debug variables and expressions
- `lookup`: A plugin to access data from outside sources

Here's an example of the `stat` command, which retrieves facts about directories:

```
1  ansible localhost -m stat -a "path=/ansible"
```

Next steps for your learning

Congratulations on taking your first steps with Ansible! This popular tool is perfect for modern DevOps environments and is easy to learn for existing developers. You're now ready to dive deeper into Ansible and learn more about topics such as:

- Ansible inventories
- Ansible roles
- Ansible containers
- And more

To help you develop your Ansible skills fast, check out Educative's course **Ansible: Zero to Production Ready**. In this self-paced, curated course, you'll learn how to set up a Docker environment, connect to the cloud, manage infrastructure within the cloud, and automate configuration and state management processes with Ansible.

By the end of the course, you'll earn a valuable certificate and learn a new skill to put on your resume that will open many doors for you in areas like DevOps and cloud computing.

Happy learning!

Continue reading about DevOps

- YAML Tutorial: get started with YAML in 5 minutes
- Docker Compose Tutorial: advanced Docker made simple